# TruthLens — Full Project Build Prompt for AI Agent

## Version: 2.0 | Stack: Next.js 16 · Python FastAPI · LangGraph · Groq/Ollama · BoldKit + shadcn/ui

---

## AGENT PRIME DIRECTIVE

You are building **TruthLens** — a disaster video misinformation detection platform. You will build the full production-ready project from scratch following every instruction below **exactly**. Read this entire prompt before writing a single line of code. Commit at every logical checkpoint as specified. Do not skip any section.

---

## SECTION 1 — REPOSITORY & PACKAGE MANAGER RULES

### 1.1 Package Manager Setup

- Use **pnpm** for local development (all `install`, `add`, `dev` commands use `pnpm`)
- The `package.json` must use **npm-compatible lockfile and scripts** so teammates using npm can run `npm install` and `npm run dev` without issues
- Add `"packageManager": "pnpm@9"` to `package.json`
- Add a `.npmrc` file with `shamefully-hoist=true` for compatibility
- In the GitHub Actions CI, use **npm** (not pnpm) explicitly for all install and run steps

### 1.2 Git Setup

Every logical unit of work must be committed with a descriptive message. Required commit points (minimum):

```
git init
git commit -m "chore: initial Next.js project scaffold"
git commit -m "chore: add shadcn/ui + boldkit theme"
git commit -m "chore: configure biome linter and formatter"
git commit -m "chore: add CI/CD GitHub Actions pipeline"
git commit -m "feat: add environment config with online/offline toggle"
git commit -m "feat: add layout, navbar, footer with boldkit components"
git commit -m "feat: add home page with video submission"
git commit -m "feat: add analysis page with agent progress UI"
git commit -m "feat: add incidents page"
git commit -m "feat: add bulletin board page"
git commit -m "feat: add community feed and voting"
git commit -m "feat: add i18n with react-i18next + Groq dynamic translation"
git commit -m "feat: add demo/real mode toggle"
git commit -m "feat: Python backend scaffold with FastAPI"
git commit -m "feat: add LangGraph agent pipeline"
git commit -m "feat: add DeepFake detector agent (online + local)"
git commit -m "feat: add Source Hunter agent"
git commit -m "feat: add Context Analyser agent"
git commit -m "feat: add Orchestrator node with LangSmith tracing"
git commit -m "feat: wire Next.js API routes to Python backend"
git commit -m "chore: add Docker setup for local AI stack"
git commit -m "docs: add README with full setup guide"
```

---

## SECTION 2 — PROJECT STRUCTURE

Create this exact directory structure:

```
truthlens/
├── .github/
|   ├── workflows/
|   |   ├── ci.yml                    # Main CI: lint, build, type-check, tests
|   |   ├── security.yml              # Dependency audit + vulnerability scan
|   |   ├── python-checks.yml         # Python lint, type-check, tests
|   |   └── deploy.yml                # Deploy to Vercel on main merge
|   └── scripts/
|       └── langgraph_api.py          # LangSmith deployment helper
├── frontend/                         # Next.js app
|   ├── src/
|   |   ├── app/
|   |   |   ├── layout.tsx
|   |   |   ├── page.tsx              # Home
|   |   |   ├── globals.css           # BoldKit + shadcn CSS vars ONLY
|   |   |   ├── analysis/
|   |   |   |   └── page.tsx
|   |   |   ├── incidents/
|   |   |   |   └── page.tsx
|   |   |   ├── bulletin/
|   |   |   |   └── page.tsx
|   |   |   ├── how-it-works/
|   |   |   |   └── page.tsx
|   |   |   └── api/
|   |   |       ├── analyze/
|   |   |       |   └── route.ts      # Proxies to Python backend
|   |   |       ├── translate/
|   |   |       |   └── route.ts      # Groq translation (server-side)
|   |   |       └── community/
|   |   |           └── route.ts
|   |   ├── components/
|   |   |   ├── ui/                   # shadcn + boldkit components (auto-
generated)
|   |   |   ├── layout/
|   |   |   |   ├── Navbar.tsx
|   |   |   |   ├── Footer.tsx
|   |   |   |   └── ModeToggle.tsx    # Online/Offline + Demo/Real toggles
|   |   |   ├── analysis/
|   |   |   |   ├── AgentPanel.tsx
|   |   |   |   ├── VerdictCard.tsx
|   |   |   |   └── ScoreRing.tsx
|   |   |   ├── community/
|   |   |   |   └── CommunityFeed.tsx
|   |   |   └── bulletin/
|   |   |       └── BulletinBoard.tsx
|   |   ├── i18n/
```

```
│   │   │   ├── config.ts             # react-i18next setup
│   │   │   ├── provider.tsx
│   │   │   └── locales/
│   │   │       ├── en.json
│   │   │       ├── hi.json
│   │   │       ├── ta.json
│   │   │       ├── ar.json
│   │   │       └── es.json
│   │   ├── lib/
│   │   │   ├── utils.ts              # cn() and shared helpers
│   │   │   ├── config.ts             # reads env vars, exports MODE config
│   │   │   └── demoData.ts           # all demo/mock data lives here
│   │   ├── hooks/
│   │   │   ├── useAnalysis.ts
│   │   │   └── useMode.ts            # reads NEXT_PUBLIC_MODE
│   │   └── types/
│   │       └── index.ts
│   ├── components.json               # shadcn + boldkit registry
│   ├── biome.json
│   ├── next.config.ts
│   ├── package.json
│   ├── tsconfig.json
│   └── .env.local.example
├── backend/                          # Python FastAPI + LangGraph
│   ├── agents/
│   │   ├── __init__.py
│   │   ├── graph.py                  # LangGraph StateGraph definition
│   │   ├── state.py                  # AgentState TypedDict
│   │   ├── nodes/
│   │   │   ├── __init__.py
│   │   │   ├── deepfake_detector.py  # Agent 1
│   │   │   ├── source_hunter.py      # Agent 2
│   │   │   ├── context_analyser.py   # Agent 3
│   │   │   └── orchestrator.py       # Final verdict node
│   │   └── tools/
│   │       ├── __init__.py
│   │       ├── ffmpeg_tools.py       # Frame extraction, metadata
│   │       ├── ocr_tools.py          # EasyOCR / Tesseract
│   │       ├── whisper_tools.py      # Local or API transcription
│   │       ├── reverse_search.py     # Google Vision, TinEye, perceptual hash
│   │       ├── metadata_db.py        # EXIF, GPS, encoding fingerprint
│   │       └── deepsafe_client.py    # Local DeepSafe Docker client
│   ├── api/
│   │   ├── __init__.py
│   │   ├── main.py                   # FastAPI app entry point
│   │   ├── routes/
│   │   │   ├── analyze.py            # POST /analyze
```

```
│   │   │   ├── status.py            # GET /status/{job_id}
│   │   │   └── health.py            # GET /health
│   │   └── models.py                # Pydantic request/response models
│   ├── config/
│   │   ├── __init__.py
│   │   └── settings.py              # pydantic-settings, reads .env
│   ├── tests/
│   │   ├── test_agents.py
│   │   ├── test_api.py
│   │   └── conftest.py
│   ├── docker/
│   │   ├── Dockerfile.backend
│   │   └── Dockerfile.deepsafe      # DeepSafe local detection container
│   ├── pyproject.toml               # uv/poetry project file
│   ├── requirements.txt
│   └── .env.example
├── docker-compose.yml               # Full local stack
├── docker-compose.online.yml        # Online-mode stack (no local models)
├── langgraph.json                   # LangGraph Studio config
└── README.md
```

---

## SECTION 3 — FRONTEND SETUP (Next.js 16)

### 3.1 Project Initialization

```bash
pnpm create next-app@latest frontend \
  --typescript \
  --tailwind \
  --app \
  --src-dir \
  --import-alias "@/*" \
  --no-turbopack
cd frontend
```

### 3.2 BoldKit + shadcn/ui Setup

**CRITICAL:** All visual theme variables MUST come from `globals.css` and `components.json`. No hardcoded hex values anywhere in component files. All colors, radii, spacing reference CSS variables only.

**Step 1** — Create `components.json` in `frontend/`:

```json
```

```
{
  "$schema": "https://ui.shadcn.com/schema.json",
  "style": "default",
  "rsc": true,
  "tsx": true,
  "tailwind": {
    "config": "",
    "css": "src/app/globals.css",
    "baseColor": "neutral",
    "cssVariables": true
  },
  "aliases": {
    "components": "@/components",
    "utils": "@/lib/utils",
    "ui": "@/components/ui",
    "lib": "@/lib",
    "hooks": "@/hooks"
  },
  "registries": {
    "@boldkit": "https://boldkit.dev/r"
  }
}
```

**Step 2** — Install BoldKit theme and components:

```bash
# Install BoldKit styles (this sets up the CSS variables in globals.css)
pnpx shadcn@latest add https://boldkit.dev/r/styles.json

# Install BoldKit components
pnpx shadcn@latest add @boldkit/button
pnpx shadcn@latest add @boldkit/card
pnpx shadcn@latest add @boldkit/input
pnpx shadcn@latest add @boldkit/badge

# Install standard shadcn components
pnpx shadcn@latest add tabs select progress separator sheet dialog toast sonner
```

**Step 3** — `globals.css` rules:

- The file must ONLY contain CSS variables from the BoldKit `styles.json` output, `@tailwind` directives, and the `@theme inline` block
- Do NOT add any custom CSS variables for colors, fonts, or spacing that conflict with BoldKit's variable names

- Dark mode variables go in `.dark {}` block as BoldKit defines them
- Any additional animation keyframes can be added at the bottom under a comment `/* TruthLens animations */`
- Font imports go at the top before `@tailwind base`

Example structure (fill with actual BoldKit output):

```
css
```

```
/* Fonts */
@import url('...');

@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    /* === BOLDKIT VARIABLES — DO NOT EDIT MANUALLY === */
    /* Paste output from: pnpx shadcn@latest add https://boldkit.dev/r/styles.json *
    --background: <boldkit-value>;
    --foreground: <boldkit-value>;
    --card: <boldkit-value>;
    --card-foreground: <boldkit-value>;
    --popover: <boldkit-value>;
    --popover-foreground: <boldkit-value>;
    --primary: <boldkit-value>;
    --primary-foreground: <boldkit-value>;
    --secondary: <boldkit-value>;
    --secondary-foreground: <boldkit-value>;
    --muted: <boldkit-value>;
    --muted-foreground: <boldkit-value>;
    --accent: <boldkit-value>;
    --accent-foreground: <boldkit-value>;
    --destructive: <boldkit-value>;
    --destructive-foreground: <boldkit-value>;
    --border: <boldkit-value>;
    --input: <boldkit-value>;
    --ring: <boldkit-value>;
    --radius: <boldkit-value>;
    /* === END BOLDKIT VARIABLES === */

    /* TruthLens semantic aliases — reference BoldKit vars only */
    --verdict-real: var(--accent);
    --verdict-misleading: var(--destructive);
    --verdict-ai: color-mix(in srgb, var(--primary) 60%, transparent);
    --verdict-unverified: var(--muted-foreground);
  }

  .dark {
    /* === BOLDKIT DARK VARIABLES — DO NOT EDIT MANUALLY === */
    /* Paste dark mode output from BoldKit styles.json */
    /* === END BOLDKIT DARK VARIABLES === */
  }
}
```

```
/* TruthLens animations */
@keyframes fadeInUp { from { opacity:0; transform:translateY(12px); } to { opacity:1
@keyframes shimmer { 0% { background-position:-200% center; } 100% { background-posi
@keyframes scanLine { 0% { top:0; } 100% { top:100%; } }
```

### 3.3 Install Frontend Dependencies

```bash
# Core
pnpm add react-i18next i18next i18next-browser-languagedetector
pnpm add lucide-react framer-motion clsx next-themes
pnpm add zod react-hook-form @hookform/resolvers
pnpm add @tanstack/react-query
pnpm add swr

# Dev
pnpm add -D @biomejs/biome @types/node
```

### 3.4 Biome Configuration

Create `frontend/biome.json`:

```json
```

```json
{
  "$schema": "https://biomejs.dev/schemas/latest/schema.json",
  "vcs": {
    "enabled": true,
    "clientKind": "git",
    "useIgnoreFile": true
  },
  "files": {
    "ignoreUnknown": false,
    "includes": ["**", "!node_modules", "!.next", "!dist", "!src/components/ui/**"]
  },
  "formatter": {
    "enabled": true,
    "indentStyle": "space",
    "indentWidth": 2,
    "lineWidth": 100
  },
  "linter": {
    "enabled": true,
    "rules": {
      "recommended": true,
      "correctness": {
        "noUnusedVariables": "error",
        "noUnusedImports": "error"
      },
      "style": {
        "useConst": "warn",
        "noNonNullAssertion": "warn"
      },
      "suspicious": {
        "noConsoleLog": "warn"
      }
    }
  },
  "javascript": {
    "formatter": {
      "quoteStyle": "single",
      "jsxQuoteStyle": "double",
      "trailingCommas": "es5"
    }
  },
  "overrides": [
    {
      "includes": ["*.test.ts", "*.test.tsx", "*.spec.ts"],
      "linter": {
        "rules": {
```

```
          "suspicious": { "noConsoleLog": "off" }
        }
      }
    }
  ]
}
```

Add to `package.json` scripts:

```json
{
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "type-check": "tsc --noEmit",
    "lint": "biome lint .",
    "lint:fix": "biome lint --write .",
    "format": "biome format --write .",
    "check": "biome check .",
    "check:fix": "biome check --write .",
    "check:ci": "biome ci ."
  }
}
```

Disable Next.js built-in ESLint in `next.config.ts`:

```ts
const nextConfig = {
  eslint: { ignoreDuringBuilds: true },
};
```

---

## SECTION 4 — ENVIRONMENT CONFIGURATION (ONLINE/OFFLINE TOGGLE)

### 4.1 `.env.local.example` (frontend)

```env
```

```
# ============================================================
# TruthLens Environment Configuration
# Copy to .env.local and fill in your values
# ============================================================

# ── MODE CONTROL ──────────────────────────────────────────
# INFERENCE_MODE: "online" | "offline"
#   online  → Uses Groq API for LLM, Hive AI for deepfake, Whisper API for transcrip
#   offline → Uses Ollama (local LLM), DeepSafe (local Docker), local Whisper model
NEXT_PUBLIC_INFERENCE_MODE=online

# APP_MODE: "demo" | "real"
#   demo → Uses sample YouTube/Instagram videos and mock agent results
#   real → Runs full analysis pipeline against submitted video
NEXT_PUBLIC_APP_MODE=demo

# ── BACKEND ───────────────────────────────────────────────
NEXT_PUBLIC_BACKEND_URL=http://localhost:8000

# ── ONLINE MODE API KEYS ──────────────────────────────────
# Groq — Free tier: https://console.groq.com
# Used for: Orchestrator LLM, translation, context analysis
GROQ_API_KEY=

# Hive AI — Free tier (100 req/day): https://thehive.ai
# Used for: DeepFake detection (online mode)
HIVE_API_KEY=

# LangSmith — Free tier: https://smith.langchain.com
# Used for: Agent tracing, observability, debugging
LANGSMITH_API_KEY=
LANGSMITH_PROJECT=truthlens
LANGSMITH_TRACING_V2=true

# Together AI — Optional alternative LLM: https://api.together.xyz
TOGETHER_API_KEY=

# ── ONLINE SOURCE HUNTING ─────────────────────────────────
# Google Vision API (for reverse image search of keyframes)
# Get from: https://console.cloud.google.com/apis/library/vision.googleapis.com
GOOGLE_VISION_API_KEY=

# TinEye API (for reverse image/video search)
# Get from: https://services.tineye.com/TinEyeAPI
TINEYE_API_KEY=
```

```
# YouTube Data API v3 (for fetching video metadata)
# Get from: https://console.cloud.google.com/apis/library/youtube.googleapis.com
YOUTUBE_API_KEY=

# Shodan API (for IP/infrastructure metadata on video sources)
# Get from: https://account.shodan.io
SHODAN_API_KEY=

# ── OFFLINE MODE CONFIG ──────────────────────────────────────
# Ollama base URL (default: http://localhost:11434)
OLLAMA_BASE_URL=http://localhost:11434
# Ollama model for orchestrator (recommend: llama3.3 or mistral)
OLLAMA_ORCHESTRATOR_MODEL=llama3.3
# Ollama vision model for frame analysis
OLLAMA_VISION_MODEL=llava:13b

# DeepSafe local API (Docker container)
DEEPSAFE_URL=http://localhost:8001

# Local Whisper model size: tiny | base | small | medium | large
WHISPER_MODEL_SIZE=medium

# ── DATABASE (optional for community features) ───────────────
# Supabase for community posts, votes, bulletins
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
```

### 4.2 Backend `.env.example`

```env
```

```
# ── MODE ──────────────────────────────────────────────────
INFERENCE_MODE=online   # online | offline
APP_MODE=demo           # demo | real

# ── ONLINE LLM ────────────────────────────────────────────
GROQ_API_KEY=
GROQ_ORCHESTRATOR_MODEL=llama-3.3-70b-versatile
GROQ_FAST_MODEL=llama-3.1-8b-instant

# ── OFFLINE LLM ───────────────────────────────────────────
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_ORCHESTRATOR_MODEL=llama3.3
OLLAMA_VISION_MODEL=llava:13b

# ── LANGSMITH TRACING ─────────────────────────────────────
LANGSMITH_API_KEY=
LANGSMITH_PROJECT=truthlens
LANGSMITH_TRACING_V2=true

# ── DEEPFAKE DETECTION ────────────────────────────────────
HIVE_API_KEY=           # Online mode
DEEPSAFE_URL=http://localhost:8001   # Offline mode

# ── WHISPER TRANSCRIPTION ─────────────────────────────────
WHISPER_USE_API=true    # true = OpenAI Whisper API, false = local model
OPENAI_API_KEY=         # Only needed if WHISPER_USE_API=true
WHISPER_MODEL_SIZE=medium  # For local: tiny|base|small|medium|large

# ── SOURCE HUNTING ────────────────────────────────────────
GOOGLE_VISION_API_KEY=
TINEYE_API_KEY=
YOUTUBE_API_KEY=

# ── METADATA DATABASES ────────────────────────────────────
# EXIF + GPS extraction is always local (ExifTool)
# No API key needed for basic metadata

# ── SERVER ────────────────────────────────────────────────
HOST=0.0.0.0
PORT=8000
```

### 4.3 Mode Config Module (`src/lib/config.ts`)

```typescript
```

```typescript
// src/lib/config.ts
// Single source of truth for runtime mode configuration
// All components read from here — never read env vars directly in components

export const config = {
  // "online" | "offline"
  inferenceMode: process.env.NEXT_PUBLIC_INFERENCE_MODE as 'online' | 'offline' ?? '

  // "demo" | "real"
  appMode: process.env.NEXT_PUBLIC_APP_MODE as 'demo' | 'real' ?? 'demo',

  backendUrl: process.env.NEXT_PUBLIC_BACKEND_URL ?? 'http://localhost:8000',

  isDemo: process.env.NEXT_PUBLIC_APP_MODE === 'demo',
  isOnline: process.env.NEXT_PUBLIC_INFERENCE_MODE === 'online',
  isOffline: process.env.NEXT_PUBLIC_INFERENCE_MODE === 'offline',
} as const;
```

### 4.4 Demo/Real Mode Toggle Component

Add a visible toggle in the Navbar that reads from `config.appMode`. In demo mode, use sample videos (see Section 8). In real mode, the submission runs through the full backend pipeline.

The toggle must:

- Show current mode in the UI (badge or pill)
- Allow switching between demo/real by updating a client-side state (no page reload needed for demo data swap)
- Show a warning banner if `NEXT_PUBLIC_APP_MODE=real` and `NEXT_PUBLIC_INFERENCE_MODE=offline` that says "Offline mode — local models required"
- Be built with BoldKit `@boldkit/badge` and `@boldkit/button` components

---

## SECTION 5 — DEMO MODE: SAMPLE VIDEOS

### 5.1 Demo Video Sources (`src/lib/demoData.ts`)

The demo mode uses real publicly accessible YouTube/Instagram videos. These are pre-analysed with static results so the demo runs without any API calls.

```typescript
```

```
export const DEMO_VIDEOS = [
  {
    id: 'demo-real-001',
    label: 'Real — Chennai Flooding (2023)',
    url: 'https://www.youtube.com/watch?v=EXAMPLE_REAL_1',
    thumbnail: 'https://img.youtube.com/vi/EXAMPLE_REAL_1/maxresdefault.jpg',
    platform: 'youtube',
    precomputedResult: DEMO_RESULT_REAL,
  },
  {
    id: 'demo-misleading-001',
    label: 'Misleading — Recirculated Flood Video',
    url: 'https://www.youtube.com/watch?v=EXAMPLE_MISLEADING_1',
    thumbnail: 'https://img.youtube.com/vi/EXAMPLE_MISLEADING_1/maxresdefault.jpg',
    platform: 'youtube',
    precomputedResult: DEMO_RESULT_MISLEADING,
  },
  {
    id: 'demo-ai-001',
    label: 'AI Generated — Fake Disaster Footage',
    url: 'https://www.instagram.com/p/EXAMPLE_AI_1/',
    thumbnail: '/demo/thumbnails/ai-generated-1.jpg',
    platform: 'instagram',
    precomputedResult: DEMO_RESULT_AI_GENERATED,
  },
];
```

Replace `EXAMPLE_*` placeholders with actual YouTube video IDs of:

1. A verified real disaster video (use official news channels — BBC, Reuters, etc.)
2. A video that was known to be shared with false context
3. A video that was flagged as AI-generated

Pre-compute realistic mock `AgentResult[]` and `AnalysisResult` for each and store in `demoData.ts`. The demo simulation should have staggered timing (3s, 5s, 7s) showing the agents completing one by one, just like the real pipeline would.

---

## SECTION 6 — GITHUB ACTIONS CI/CD

### 6.1 Main CI (`ci.yml`)

```yaml
```

```yaml
name: CI

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main, develop]

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  frontend-checks:
    name: Frontend — Lint, Type Check, Build
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
          cache-dependency-path: frontend/package-lock.json

      - name: Install dependencies
        run: npm install

      - name: Biome CI check
        uses: biomejs/setup-biome@v2
        with:
          version: latest
      - run: biome ci .

      - name: TypeScript type check
        run: npm run type-check

      - name: Next.js build
        run: npm run build
        env:
          NEXT_PUBLIC_INFERENCE_MODE: online
```

```
        NEXT_PUBLIC_APP_MODE: demo
        NEXT_PUBLIC_BACKEND_URL: http://localhost:8000
```

### 6.2 Security Checks (`security.yml`)

yaml

```yaml
name: Security

on:
  push:
    branches: [main]
  pull_request:
  schedule:
    - cron: '0 6 * * 1'   # Weekly Monday audit

jobs:
  npm-audit:
    name: npm audit
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: frontend
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - run: npm install
      - run: npm audit --audit-level=moderate

  dependency-review:
    name: Dependency Review (PRs only)
    runs-on: ubuntu-latest
    if: github.event_name == 'pull_request'
    steps:
      - uses: actions/checkout@v4
      - uses: actions/dependency-review-action@v4

  codeql:
    name: CodeQL Analysis
    runs-on: ubuntu-latest
    permissions:
      actions: read
      contents: read
      security-events: write
    strategy:
      matrix:
        language: [javascript, python]
    steps:
      - uses: actions/checkout@v4
      - uses: github/codeql-action/init@v3
        with:
```

```yaml
        languages: ${{ matrix.language }}
    - uses: github/codeql-action/autobuild@v3
    - uses: github/codeql-action/analyze@v3
```

### 6.3 Python Checks (`python-checks.yml`)

```yaml
```

```yaml
name: Python Checks

on:
  push:
    paths: ['backend/**']
  pull_request:
    paths: ['backend/**']

jobs:
  python-quality:
    name: Python — Lint, Type Check, Tests
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: backend

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: pip

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Ruff lint
        run: ruff check .

      - name: Ruff format check
        run: ruff format --check .

      - name: Mypy type check
        run: mypy . --ignore-missing-imports

      - name: Pytest
        run: pytest tests/ -v --tb=short
        env:
          INFERENCE_MODE: demo
          APP_MODE: demo
```

### 6.4 Deploy (`deploy.yml`)

yaml

```yaml
name: Deploy

on:
  push:
    branches: [main]

jobs:
  deploy-frontend:
    name: Deploy to Vercel
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - working-directory: frontend
        run: npm install
      - uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${{ secrets.VERCEL_TOKEN }}
          vercel-org-id: ${{ secrets.VERCEL_ORG_ID }}
          vercel-project-id: ${{ secrets.VERCEL_PROJECT_ID }}
          working-directory: frontend
          vercel-args: '--prod'
```

---

## SECTION 7 — PYTHON BACKEND (FastAPI + LangGraph)

### 7.1 Dependencies (`requirements.txt`)

```python
# Core
fastapi==0.115.0
uvicorn[standard]==0.30.0
pydantic==2.7.0
pydantic-settings==2.3.0
python-multipart==0.0.9

# LangChain + LangGraph + LangSmith
langchain==1.2.0
langgraph==0.2.0
langchain-groq==0.1.9
langchain-community==0.2.0
langsmith==0.1.0
```

```
# Video processing
ffmpeg-python==0.2.0
opencv-python-headless==4.10.0.82
imagehash==4.3.1
Pillow==10.4.0
exiftool==0.5.6

# Audio
openai-whisper==20231117    # local Whisper
openai==1.35.0              # Whisper API fallback

# OCR
easyocr==1.7.1

# HTTP
httpx==0.27.0
aiofiles==23.2.1

# Dev
pytest==8.2.0
pytest-asyncio==0.23.0
ruff==0.5.0
mypy==1.10.0
```

### 7.2 Settings (`backend/config/settings.py`)

```python
```

```python
from pydantic_settings import BaseSettings
from typing import Literal

class Settings(BaseSettings):
    # Mode
    inference_mode: Literal["online", "offline"] = "online"
    app_mode: Literal["demo", "real"] = "demo"

    # Online LLM
    groq_api_key: str = ""
    groq_orchestrator_model: str = "llama-3.3-70b-versatile"
    groq_fast_model: str = "llama-3.1-8b-instant"

    # Offline LLM
    ollama_base_url: str = "http://localhost:11434"
    ollama_orchestrator_model: str = "llama3.3"
    ollama_vision_model: str = "llava:13b"

    # LangSmith
    langsmith_api_key: str = ""
    langsmith_project: str = "truthlens"
    langsmith_tracing_v2: str = "true"

    # Deepfake
    hive_api_key: str = ""
    deepsafe_url: str = "http://localhost:8001"

    # Whisper
    whisper_use_api: bool = True
    openai_api_key: str = ""
    whisper_model_size: str = "medium"

    # Source hunting
    google_vision_api_key: str = ""
    tineye_api_key: str = ""
    youtube_api_key: str = ""

    # Server
    host: str = "0.0.0.0"
    port: int = 8000

    class Config:
        env_file = ".env"
        case_sensitive = False

settings = Settings()
```

```python
def get_llm():
    """Returns the appropriate LLM based on INFERENCE_MODE."""
    if settings.inference_mode == "offline":
        from langchain_community.llms import Ollama
        return Ollama(
            base_url=settings.ollama_base_url,
            model=settings.ollama_orchestrator_model,
        )
    else:
        from langchain_groq import ChatGroq
        return ChatGroq(
            api_key=settings.groq_api_key,
            model=settings.groq_orchestrator_model,
        )
```

### 7.3 LangGraph Agent State (`backend/agents/state.py`)

```python
```

```python
from typing import TypedDict, List, Optional, Literal
from dataclasses import dataclass, field

@dataclass
class AgentFinding:
    agent_id: str
    status: Literal["idle", "running", "done", "error"]
    score: Optional[float]
    findings: List[str]
    detail: Optional[str]

class AgentState(TypedDict):
    # Input
    video_url: Optional[str]
    video_path: Optional[str]
    job_id: str

    # Intermediate — populated by each agent node
    keyframes: List[str]        # Paths to extracted frames
    audio_path: Optional[str]   # Path to extracted audio
    metadata: dict              # EXIF, GPS, encoding info
    transcript: Optional[str]   # Whisper output
    ocr_text: Optional[str]     # On-screen text

    # Agent results
    deepfake_result: Optional[AgentFinding]
    source_result: Optional[AgentFinding]
    context_result: Optional[AgentFinding]

    # Final
    verdict: Optional[str]
    credibility_score: Optional[int]
    panic_index: Optional[int]
    summary: Optional[str]
    error: Optional[str]
```

### 7.4 LangGraph Graph Definition (`backend/agents/graph.py`)

```python
```

```python
import asyncio
from langgraph.graph import StateGraph, END
from langsmith import traceable
from .state import AgentState
from .nodes.deepfake_detector import deepfake_detector_node
from .nodes.source_hunter import source_hunter_node
from .nodes.context_analyser import context_analyser_node
from .nodes.orchestrator import orchestrator_node
from ..tools.ffmpeg_tools import extract_keyframes, extract_audio

def create_truthlens_graph():
    """
    TruthLens LangGraph pipeline.

    Flow:
      [preprocess] → [deepfake, source, context] (parallel fan-out) → [orchestrator]

    The three detection agents run concurrently using asyncio.gather inside
    a single 'parallel_analysis' node (LangGraph doesn't natively support
    fan-out; we simulate it with concurrent async calls and merge results).
    """
    workflow = StateGraph(AgentState)

    # Node: Pre-process video (extract frames + audio)
    workflow.add_node("preprocess", preprocess_node)

    # Node: Run all 3 agents concurrently
    workflow.add_node("parallel_analysis", parallel_analysis_node)

    # Node: Orchestrator compiles final verdict
    workflow.add_node("orchestrator", orchestrator_node)

    # Edges
    workflow.set_entry_point("preprocess")
    workflow.add_edge("preprocess", "parallel_analysis")
    workflow.add_edge("parallel_analysis", "orchestrator")
    workflow.add_edge("orchestrator", END)

    return workflow.compile()

@traceable(name="preprocess")
async def preprocess_node(state: AgentState) -> AgentState:
    """Extract keyframes and audio from the video."""
    frames = await extract_keyframes(state.get("video_path") or state.get("video_url
    audio = await extract_audio(state.get("video_path") or state.get("video_url"))
    return {**state, "keyframes": frames, "audio_path": audio}
```

```python
@traceable(name="parallel_analysis")
async def parallel_analysis_node(state: AgentState) -> AgentState:
    """Run all 3 detection agents concurrently and merge results."""
    deepfake_result, source_result, context_result = await asyncio.gather(
        deepfake_detector_node(state),
        source_hunter_node(state),
        context_analyser_node(state),
        return_exceptions=False,
    )
    return {
        **state,
        "deepfake_result": deepfake_result,
        "source_result": source_result,
        "context_result": context_result,
    }

# Instantiate once at import
graph = create_truthlens_graph()
```

### 7.5 Agent Node Implementation Rules

Each agent node file must follow this pattern. Show full implementation for all three:

#### Agent 1: DeepFake Detector (`nodes/deepfake_detector.py`)

```python
```

```
# Online mode: POST frames to Hive AI API
# Offline mode: POST frames to DeepSafe local Docker API at DEEPSAFE_URL
# Fallback: Basic pixel variance heuristic (always available)

# Online mode (Hive AI):
# POST https://api.thehive.ai/api/v2/task/sync
# Headers: token: {HIVE_API_KEY}
# Body: multipart form with image file
# Response: {"status": [{"response": {"output": [{"time": 0, "classes": [{"class": "

# Offline mode (DeepSafe):
# DeepSafe runs in Docker on port 8001
# POST http://localhost:8001/api/detect
# Body: {"image_base64": "...", "model": "CrossEfficientViT"}
# Response: {"is_fake": true, "confidence": 0.94, "model_used": "CrossEfficientViT"}

# For video: extract keyframes first, analyse each frame, take max confidence

async def deepfake_detector_node(state: AgentState) -> AgentFinding:
    from config.settings import settings
    if settings.inference_mode == "offline":
        return await _deepsafe_detect(state)
    else:
        return await _hive_detect(state)
```

### Agent 2: Source Hunter (`nodes/source_hunter.py`)

```python
```

```
# Online mode:
#   1. Extract 5 keyframes from video
#   2. Compute perceptual hash (pHash) for each frame using imagehash library
#   3. POST each keyframe to Google Vision API for web entity detection
#      GET https://vision.googleapis.com/v1/images:annotate?key={GOOGLE_VISION_API_K
#      Body: {"requests": [{"image": {"content": "<base64>"}, "features": [{"type":
#      Response gives: webEntities, fullMatchingImages, partialMatchingImages, pages
#   4. Also POST to TinEye API:
#      POST https://api.tineye.com/rest/search/?api_key={TINEYE_API_KEY}
#      Response gives: matches with domains and first_seen dates
#   5. Extract EXIF metadata: GPS coords, creation timestamp, encoding software
#      Use exiftool subprocess: subprocess.run(["exiftool", "-json", video_path])
#   6. If YouTube URL: GET https://www.googleapis.com/youtube/v3/videos?id={vid}&key
#      Extract: publishedAt, channelTitle, description, recordingDate, location

# Offline mode:
#   Steps 1-2 (pHash) — always available, no API needed
#   Step 5 (EXIF) — always available via exiftool
#   Steps 3-4 skip (require API), log warning
#   Step 6 skips (requires API)

# The source hunter returns:
#   - earliest known source (URL + date)
#   - list of re-uploads with different claims
#   - GPS coordinates if available
#   - encoding fingerprint suspicious flags
```

### Agent 3: Context Analyser (`nodes/context_analyser.py`)

```python
```

```
# Online mode:
#   1. Transcribe audio:
#      POST https://api.openai.com/v1/audio/transcriptions
#      File: audio.wav, model: whisper-1, response_format: verbose_json
#      Returns: text, language, segments with timestamps
#   2. OCR on keyframes using EasyOCR:
#      reader = easyocr.Reader(['en', 'hi', 'ta', 'ar'])
#      results = reader.readtext(frame_path)
#   3. Detect language of text and audio
#   4. Send keyframes + transcript + OCR to vision LLM (Groq llama-3.2-11b-vision or
#      Prompt: "Analyse this disaster video frame. Identify: country, city, language
#               weather conditions, vehicle types, architecture style, any text visi
#               Does the content match the claimed location: {claimed_location}?
#               Return JSON: {location_match: bool, confidence: 0-100, evidence: [..
#   5. Check for audio manipulation: compare audio spectrum for added siren/music ov
#      using librosa: librosa.feature.spectral_centroid, librosa.effects.split

# Offline mode:
#   1. Local Whisper: whisper.load_model("medium").transcribe(audio_path)
#   2. EasyOCR: same as online (no API)
#   3. Vision LLM: Ollama with llava:13b model
#      import ollama; ollama.chat(model='llava:13b', messages=[...])
#   4. Audio analysis: librosa (always local)
```

### 7.6 Orchestrator Node (`nodes/orchestrator.py`)

```python
```

```
# The orchestrator receives all three AgentFinding results and:
# 1. Calls the LLM (Groq online / Ollama offline) with the structured findings
# 2. Gets back: verdict, credibility_score, panic_index, summary
# 3. LangSmith traces this call automatically via LANGSMITH_TRACING_V2=true

# Prompt to LLM:
ORCHESTRATOR_PROMPT = """
You are the TruthLens Orchestrator. Three AI agents have analysed a disaster video.
Your job is to synthesise their findings into a final public verdict.

AGENT RESULTS:
{agent_results_json}

Produce a verdict. Respond ONLY with valid JSON (no markdown):
{{
  "verdict": "real" | "misleading" | "ai-generated" | "unverified",
  "credibility_score": <0-100>,
  "panic_index": <0-10>,
  "summary": "<2-3 sentence plain English verdict for the public>",
  "source_origin": "<earliest known source if found>",
  "original_date": "<date if found>",
  "claimed_location": "<claimed location>",
  "actual_location": "<confirmed location if different>",
  "key_flags": ["<flag1>", "<flag2>"]
}}
"""
```

---

## SECTION 8 — ONLINE METADATA DATABASES TO INTEGRATE

Connect these external databases/APIs for richer signal in production (real mode):

### Source Verification

| Database | What It Provides | Integration Point | Free? |
|---|---|---|---|
| **Google Vision Web Detection** | Reverse image search, matching web pages, earliest instance | Source Hunter | 1000 req/month free |
| **TinEye API** | Image reverse search, first seen date, domain history | Source Hunter | 150 req/month free |
| **YouTube Data API v3** | Video metadata, channel, upload date, geo-tag | Source Hunter | Free with key |

| Database | What It Provides | Integration Point | Free? |
|---|---|---|---|
| **Wayback Machine API** | Check if URL was archived before claimed date | Source Hunter | Free |

### Geolocation & Context

| Database | What It Provides | Integration Point | Free? |
|---|---|---|---|
| **OpenStreetMap Nominatim** | GPS coords → place name, country | Context Analyser | Free |
| **Open-Meteo API** | Historical weather for claimed date/location | Context Analyser | Free |
| **GeoNames** | City/country lookup from GPS | Context Analyser | Free |

### AI Detection

| Database | What It Provides | Integration Point | Free? |
|---|---|---|---|
| **Hive AI** | Deepfake detection (image/video/audio) | DeepFake Detector | 100 req/day free |
| **AI or Not API** | Quick AI image detection | DeepFake Detector (fallback) | Free tier |

### Fact-Checking

| Database | What It Provides | Integration Point | Orchestrator |
|---|---|---|---|
| **Google Fact Check Tools API** | Existing fact checks for claims | Orchestrator | Free |
| **ClaimBuster API** | Claim detection and check-worthiness | Context Analyser | Free |

Integrate all free-tier APIs as passive enrichment — if they fail or rate-limit, the analysis continues without them. Each API call must be wrapped in `try/except` with a 5-second timeout.

---

## SECTION 9 — LOCAL AI DETECTOR SETUP (OFFLINE MODE)

Provide these exact setup instructions inside the README.md under a "Local AI Setup" section:

### DeepSafe (Local Deepfake Detection)

DeepSafe runs as a Docker container exposing an API at port 8001.

```bash
# Clone DeepSafe
git clone https://github.com/siddharthksah/DeepSafe
cd DeepSafe

# Run with Docker Compose (includes CrossEfficientViT for video)
docker compose up -d

# Test it's running
curl http://localhost:8001/health
```

The `docker-compose.yml` in TruthLens exposes this internally and the agent uses `DEEPSAFE_URL=http://deepsafe:8001`.

Models included in DeepSafe:

- **CrossEfficientViT** — Video deepfake detection (primary for TruthLens)
- **UniversalFakeDetect** — Image AI generation detection
- **NPR** — Image deepfake detection

### Ollama (Local LLM)

```bash
# Install Ollama
curl -fsSL https://ollama.ai/install.sh | sh

# Pull models
ollama pull llama3.3          # Orchestrator (8B, ~5GB)
ollama pull llava:13b          # Vision model for frame analysis (~8GB)
ollama pull llama3.1:8b        # Fast model for translation
```

Minimum hardware for offline mode:

- 16GB RAM (32GB recommended for llava:13b)
- NVIDIA GPU with 8GB+ VRAM for acceptable speed
- CPU-only is possible but slow (5-15min per analysis)

### Local Whisper

```bash
pip install openai-whisper

# Test
python -c "import whisper; m=whisper.load_model('medium'); print('Whisper ready')"
```

Model sizes (choose based on hardware):

- `tiny` — fastest, lower accuracy (~1GB)
- `base` — good balance for simple speech (~1GB)
- `small` — better multilingual support (~2GB)
- `medium` — recommended default (~5GB)
- `large` — highest accuracy, slowest (~10GB)

### Docker Compose (Full Offline Stack)

`docker-compose.yml`:

```yaml
```

```yaml
version: '3.9'
services:
  backend:
    build:
      context: ./backend
      dockerfile: docker/Dockerfile.backend
    ports:
      - "8000:8000"
    environment:
      - INFERENCE_MODE=${INFERENCE_MODE:-online}
      - APP_MODE=${APP_MODE:-demo}
      - GROQ_API_KEY=${GROQ_API_KEY}
      - OLLAMA_BASE_URL=http://ollama:11434
      - DEEPSAFE_URL=http://deepsafe:8001
    env_file: ./backend/.env
    depends_on:
      - ollama
      - deepsafe

  ollama:
    image: ollama/ollama:latest
    ports:
      - "11434:11434"
    volumes:
      - ollama_data:/root/.ollama
    deploy:
      resources:
        reservations:
          devices:
            - driver: nvidia
              count: all
              capabilities: [gpu]

  deepsafe:
    build:
      context: ./backend/docker
      dockerfile: Dockerfile.deepsafe
    ports:
      - "8001:8001"

volumes:
  ollama_data:
```

---

## SECTION 10 — FRONTEND PAGES & COMPONENTS

### 10.1 Component Rules (MANDATORY)

- Every component must use BoldKit components as base: `@boldkit/button`, `@boldkit/card`, `@boldkit/input`, `@boldkit/badge`
- Style overrides use Tailwind utility classes only — no inline `style={{}}` with hex values
- Colors reference CSS variables via Tailwind: `bg-primary`, `text-muted-foreground`, `border-destructive`, etc.
- Custom variants extend the BoldKit component variants using `cva` — don't re-create from scratch
- The `cn()` utility from `@/lib/utils` is used for all className merging

### 10.2 Pages to Build (all as described in the spec doc)

1. **Home** (`/`) — Video URL input + file upload, demo/real toggle, active incidents list, platform stats, how-it-works preview
2. **Analysis** (`/analysis`) — Live agent progress, verdict panel with score rings, expandable technical report, community feed, bulletin, share card
3. **Incidents** (`/incidents`) — Filterable grid of all incidents with misinfo rate bars
4. **Bulletin** (`/bulletin`) — Auto-generated incident bulletin with confirmed/debunked/unverified/official sections
5. **How It Works** (`/how-it-works`) — Visual pipeline explainer, tech stack, local AI setup guide

### 10.3 API Routes (Next.js)

- `POST /api/analyze` — If `APP_MODE=demo`, return precomputed demo result with simulated streaming delay. If `APP_MODE=real`, proxy to `BACKEND_URL/analyze`
- `POST /api/translate` — Always call Groq API server-side. If `INFERENCE_MODE=offline`, call Ollama translation endpoint
- `GET /api/incidents` — Returns `MOCK_INCIDENTS` in demo mode, real DB in production

---

## SECTION 11 — i18n STRATEGY

### Static UI (react-i18next hardcoded JSON)

- All labels, buttons, headings, nav items
- Languages: `en`, `hi`, `ta`, `ar`, `es`
- Auto-detect from `navigator.language` on mount

- JSON files in `src/i18n/locales/`
- Use `useTranslation()` hook in all 'use client' components

**Dynamic Content (Groq / Ollama translation via API)**

- Community posts, bulletin items, analysis summaries
- Called server-side via `/api/translate`
- Online mode: Groq `llama-3.1-8b-instant`
- Offline mode: Ollama `llama3.1:8b`
- Cache results in memory (Map) keyed by `${lang}:${hash(text)}`
- Show "Translating via Groq..." / "Translating locally..." indicator

---

## SECTION 12 — README.md

The README must include these sections (write them fully):

1. **What is TruthLens** — One paragraph project description
2. **Architecture Overview** — Diagram (text-based) showing Next.js → FastAPI → LangGraph → Agents → External APIs
3. **Quick Start (Demo Mode)** — 5 commands to get running with zero API keys
4. **Environment Setup** — Full table of all env vars, what they do, where to get them
5. **Online Mode Setup** — List of API keys needed, where to get each (with direct links), free tier limits
6. **Offline Mode Setup** — Step-by-step: DeepSafe Docker, Ollama pull, local Whisper
7. **Switching Between Modes** — Exactly which env vars to change for each combination:
    - Demo + Online (default, zero config)
    - Demo + Offline (hackathon, no internet)
    - Real + Online (production)
    - Real + Offline (air-gapped deployment)
8. **LangGraph Agent Pipeline** — Explanation of each node, state shape, tracing with LangSmith
9. **CI/CD Pipeline** — What each GitHub Action does and when it runs
10. **Contributing** — How to add new agents, new languages, new detection APIs
11. **Deployment** — Vercel (frontend) + Railway/Render (backend) + local Docker

---

## SECTION 13 — FINAL CHECKLIST

Before considering the build complete, verify all of these:

**Frontend:**

- [ ] BoldKit `styles.json` was applied and CSS vars are in `globals.css` — no hex colors in component files
- [ ] All pages build with `npm run build` (0 errors)
- [ ] `biome ci .` passes (0 errors)
- [ ] `tsc --noEmit` passes (0 errors)
- [ ] Demo mode works with no API keys set
- [ ] Real mode toggle shows correct UI state
- [ ] Language switcher works for at least EN, HI, TA, AR
- [ ] All BoldKit components (`@boldkit/button`, `@boldkit/card`, `@boldkit/input`, `@boldkit/badge`) are used somewhere visible

**Backend:**

- [ ] `ruff check .` passes
- [ ] `mypy .` passes
- [ ] `pytest` passes with demo fixtures (no real API calls in tests)
- [ ] `INFERENCE_MODE=online` uses Groq, `INFERENCE_MODE=offline` uses Ollama
- [ ] `APP_MODE=demo` returns mock data without calling any external API
- [ ] LangSmith traces appear in dashboard when `LANGSMITH_TRACING_V2=true`

**CI/CD:**

- [ ] `ci.yml` runs on every push and PR
- [ ] `security.yml` runs weekly and on PRs
- [ ] `python-checks.yml` runs only when `backend/` changes
- [ ] `deploy.yml` deploys only on `main` merge

**Git:**

- [ ] All commits from Section 1.2 are present in history
- [ ] No API keys committed (`.env` files in `.gitignore`)
- [ ] `package-lock.json` committed (for npm compatibility)

---

## CRITICAL REMINDERS

1. **BoldKit theme is the source of truth.** Run `pnpx shadcn@latest add https://boldkit.dev/r/styles.json` first. Whatever CSS variables that command outputs into `globals.css` is what you use. Do not manually write color values.

2. **Online/Offline is backend-side.** The frontend only reads `NEXT_PUBLIC_INFERENCE_MODE` to show the correct UI label. The actual switching happens in `backend/config/settings.py`'s `get_llm()` function and in each agent node's `if settings.inference_mode == "offline"` branch.
3. **Demo mode must work with zero internet.** If `APP_MODE=demo`, neither frontend nor backend should make any external API calls. All demo data is pre-computed in `src/lib/demoData.ts` and `backend/tests/fixtures/`.
4. **LangSmith traces everything automatically.** Just set `LANGSMITH_TRACING_V2=true` and `LANGSMITH_API_KEY`. Every `@traceable` decorated function and every LangChain/LangGraph call will appear in the LangSmith dashboard with full state, timing, and token counts.
5. **Commit after every section.** Do not batch multiple sections into one commit.